



TC2038 - Análisis y diseño de algoritmos avanzados

Tema 2.1 Algoritmos de Búsqueda de Patrones

Profesor - Alison Muñoz Capote
Computación - Facultad de Ingeniería

Mapa del Tema y Objetivos

Objetivos del Tema:

1. **Comprender** la ineficiencia de los algoritmos de búsqueda por fuerza bruta.
2. **Analizar** el funcionamiento del arreglo π (LPS) en el algoritmo KMP.
3. **Desglosar** el cálculo lineal de prefijos usando la Z-Function y la técnica de *Z-box*.
4. **Evaluar** la complejidad espacial y temporal de ambas aproximaciones.
5. **Implementar** soluciones prácticas.

Índice:

1. **Situación Problema:** El cuello de botella en secuencias masivas.
2. **Algoritmo KMP:** Teoría y ejemplo paso a paso.
3. **Z-Function:** Conceptos y casos de simetría.
4. Solución Problema 1 (Genómica con KMP).
5. Solución Problema 2 (Indexación con Z-Function).
6. **Tarea:** Introducción al Algoritmo de Manacher.

1. Introducción

Situación Problema: El Genoma y la Amnesia

Imaginemos que estamos desarrollando software para análisis bioinformático. Nuestro sistema recibe una secuencia de ADN mutado (el **texto**, de miles de millones de bases `A, C, G, T`) y necesitamos encontrar un gen específico responsable de una anomalía (el **patrón**, de varios miles de bases).

El procesamiento debe ser en tiempo real. Un retraso en el emparejamiento bloquea el *pipeline* de datos.

El Enfoque Ingenuo (Fuerza Bruta)

La solución inicial del equipo junior fue la fuerza bruta:

- Alinear el patrón al inicio del texto.
- Comparar carácter por carácter de izquierda a derecha.
- Si hay un fallo (mismatch), desplazar el patrón **una sola posición** a la derecha y empezar a comparar desde el carácter 0 del patrón otra vez.

¿Por qué falla la Fuerza Bruta?

El problema radica en la *amnesia algorítmica*.

Si comparamos un patrón de 10,000 bases y fallamos en la base 9,999, la fuerza bruta descarta ese progreso, avanza un solo espacio, y vuelve a empezar. Ha *olvidado* la estructura de las 9,998 bases que acaba de validar con éxito.

Consecuencia: El tiempo de ejecución se dispara a $O(n \times m)$, volviendo el sistema inoperable para grandes volúmenes de datos.

2. La búsqueda *inteligente* de patrones

Introducción al Algoritmo KMP

Diseñado por Donald Knuth, Vaughan Pratt y James H. Morris en 1977, el algoritmo KMP soluciona la amnesia algorítmica.

Su filosofía: **El propio patrón contiene la información necesaria para saber cuánto desplazarse ante un fallo.**

Analizando el patrón de antemano, podemos saltar posiciones inútiles y garantizar que el texto principal se escanee en una sola pasada $O(n)$.

Conceptos Clave: Prefijos y Sufijos Propios

Para entender KMP, debemos definir dos conceptos en una cadena (ej. `CASA`):

- **Prefijo Propio:** Subcadenas que inician en el índice 0, pero no son la cadena completa (`C`, `CA`, `CAS`).
- **Sufijo Propio:** Subcadenas que terminan en el último índice, pero no son la cadena completa (`A`, `SA`, `ASA`).

KMP busca las simetrías donde **un prefijo propio es idéntico a un sufijo propio**.

El Arreglo π (LPS)

El núcleo de KMP es el pre-procesamiento del patrón para generar el arreglo π (LPS: *Longest Proper Prefix which is also Suffix*).

Para cada posición i del patrón, el arreglo $\pi[i]$ almacena la longitud del prefijo propio más largo que también es un sufijo en la subcadena que va desde 0 hasta i .

Construcción del Arreglo π : Ejemplo

Analizaremos el patrón `P = "A B A B A C"`.

Usaremos dos punteros:

- `i` : Explora los nuevos caracteres (inicia en 1).
- `j` : Rastrea la longitud del prefijo actual (inicia en 0).

Inicializamos el arreglo `pi = [0, 0, 0, 0, 0, 0]`.

Arreglo π : Paso 1 ($i=1$)

Patrón: A B A B A C

- $i = 1$ (letra 'B')
- $j = 0$ (espera la letra 'A')
- ¿ $P[1] == P[0]$? No ('B' != 'A').
- No hay coincidencias previas ($j=0$).
- Resultado: $\pi[1] = 0$.
- Incrementamos i .

Arreglo π : Paso 2 ($i=2$)

Patrón: A B A B A C

- $i = 2$ (letra 'A')
- $j = 0$ (espera la letra 'A')
- ¿ $P[2] == P[0]$? ¡Sí! ('A' == 'A').
- El prefijo ha crecido a tamaño 1.
- Incrementamos j a 1.
- Resultado: $\pi[2] = 1$.
- Incrementamos i .

Arreglo π : Pasos 3 y 4 ($i=3, i=4$)

Paso 3 ($i=3$, letra 'B'):

¿ $P[3] == P[j]$ donde $j=1$ ('B' == 'B')? ¡Sí!
 j se vuelve 2. $\pi[3] = 2$.

Paso 4 ($i=4$, letra 'A'):

¿ $P[4] == P[j]$ donde $j=2$ ('A' == 'A')? ¡Sí!
 j se vuelve 3. $\pi[4] = 3$.

Arreglo parcial: $[0, 0, 1, 2, 3, 0]$

Arreglo π : Paso 5 (El Fallo)

Patrón: A B A B A C

- $i = 5$ (letra 'C'). $j = 3$ (espera 'B').
- ¿ $P[5] == P[3]$? No ('C' != 'B').
- Aquí KMP brilla. En lugar de hacer $j = 0$, usamos la memoria del arreglo: $j = \pi[j-1]$.
- $j = \pi[2] \rightarrow j = 1$.
- Comparamos de nuevo: ¿ $P[5] == P[1]$ ('C' == 'B')? No.
- $j = \pi[0] \rightarrow j = 0$. $\pi[5] = 0$.

Fase de Búsqueda KMP

Una vez construido el arreglo π , buscamos en el texto (puntero i) usando el patrón (puntero j):

1. Si `Texto[i] == Patrón[j]`, ambos avanzan.
2. Si fallan y `j > 0`, el texto **no retrocede**. El patrón se desliza usando: `j = pi[j-1]`.
3. Si fallan y `j == 0`, solo avanzamos el texto `i += 1`.
4. Si `j == len(Patrón)`, encontramos una coincidencia en `i - j`.

Complejidad de KMP (Tiempo)

Tiempo de Construcción (π): $O(m)$ donde m es la longitud del patrón. Aunque hay un bucle `while` interno, la variable `j` a lo largo de toda la ejecución solo puede decrementar tantas veces como incrementó (lo cual está acotado por m).

Tiempo de Búsqueda: $O(n)$ donde n es la longitud del texto principal, por el mismo principio matemático.

Complejidad Temporal Total: $O(n + m)$. Lineal estricto.

Complejidad de KMP (Espacio)

El algoritmo KMP requiere almacenar el arreglo π .
Este arreglo tiene exactamente el mismo tamaño que el patrón a buscar.

Complejidad Espacial: $O(m)$.

Para la fase de búsqueda no se requiere memoria extra aparte de variables enteras. Es extremadamente eficiente en memoria.

Implementación de π

```
def calcular_arreglo_pi(patron: str) -> list[int]:
    if not patron:
        return []

    m = len(patron)
    pi = [0] * m

    # 'j' rastrea la longitud del prefijo actual
    # y también sirve como el índice del carácter que esperamos encontrar
    j = 0

    # 'i' es el explorador que avanza siempre hacia adelante
    for i in range(1, m):
        # Si hay una discrepancia y tenemos un prefijo acumulado,
        # retrocedemos usando el "enlace de fallo" sin reiniciar 'j' a cero.
        while j > 0 and patron[i] != patron[j]:
            j = pi[j - 1]

        # Si los caracteres coinciden, el prefijo crece en 1
        if patron[i] == patron[j]:
            j += 1

        # Guardamos la longitud del prefijo válido para la posición 'i'
        pi[i] = j

    return pi
```

Introducción a la Z-Function

Mientras KMP se enfoca en *saltar* mediante prefijos-sufijos propios, la función Z tiene un enfoque ligeramente diferente.

El **Arreglo Z** de una cadena almacena en `z[i]` la longitud de la subcadena más larga que comienza en `i` y que al mismo tiempo es un prefijo de la cadena original.

Concepto Clave: El Arreglo Z

Ejemplo con `S = "a b a c a b a"`:

- `Z[0] = 0` (por definición).
- `Z[1] = 0` ('b' no coincide con el prefijo 'a').
- `Z[2] = 1` ("a" en índice 2 coincide con prefijo "a").
- `Z[4] = 3` ("a b a" en índice 4 coincide con el prefijo "a b a").

Para lograr $O(n)$, el algoritmo Z usa una **Z-box** (ventana de coincidencia).

Z-Function: Caso 1

Caso 1: Fuera de la Z-box ($i > R$)

Si nuestro índice actual i está más allá del límite derecho R que ya conocemos, no tenemos información histórica útil.

Acción: Debemos comparar de forma ingenua (fuerza bruta) los caracteres $s[i]$, $s[i+1]$ con los prefijos $s[0]$, $s[1]$.

Al terminar, si encontramos coincidencias, actualizamos nuestra nueva caja $[L, R]$.

Z-Function: Caso 2a

Caso 2a: Dentro de la caja ($i \leq R$), y la coincidencia histórica no rebasa R .

Como i está dentro de $[L, R]$, sabemos que la subcadena que empieza en i es idéntica a la que empieza en un índice anterior $k = i - L$.

Observamos el valor que ya calculamos en $z[k]$.

Acción: Si $z[k]$ es menor que la distancia hasta el borde de la caja ($R - i + 1$), entonces simplemente copiamos el valor: $z[i] = z[k]$. ¡No hay necesidad de comparar caracteres!

Z-Function: Caso 2b

Caso 2b: Dentro de la caja ($i \leq R$), pero la coincidencia histórica rebasa o toca R .

Sabemos que al menos hay coincidencias hasta el borde R . Sin embargo, más allá de R es terreno desconocido.

Acción: Asignamos $Z[i] = R - i + 1$ como valor base, y luego intentamos expandir manualmente comprobando los caracteres más allá de R . Actualizamos la caja si logramos expandir.

Complejidad de Z-Function

Complejidad Temporal: $O(n)$. Aunque parezca haber un bucle anidado, el borde derecho de la Z-box, R , nunca retrocede. Cada vez que hacemos una comparación explícita exitosa, R avanza. Como R no puede exceder n , hacemos a lo sumo n comparaciones.

Complejidad Espacial: $O(n)$ para almacenar el arreglo Z del tamaño de la cadena analizada.

Implementación de Z

```
def calcular_funcion_z(s: str) -> list[int]:
    n = len(s)
    z = [0] * n

    # L y R definen nuestra Z-box (la ventana de coincidencia)
    L, R = 0, 0

    for i in range(1, n):
        # CASO 1 Y 2a: ¿Estamos dentro de una ventana conocida?
        if i <= R:
            # Reutilizamos la información histórica.
            # Tomamos el mínimo entre lo que queda de la caja y el valor Z equivalente del pasado.
            z[i] = min(R - i + 1, z[i - L])

        # CASO 2b Y 3: Intentar expandir coincidencia hacia la derecha
        # Comparamos el prefijo (z[i]) con nuestra posición actual (i + z[i])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1

        # Actualizar los límites de la Z-box si fuimos más allá de R
        if i + z[i] - 1 > R:
            L = i
            R = i + z[i] - 1

    return z
```

3. Solución de problemas

Solución Problema 1 (KMP): Análisis

Requerimiento: Retornar los índices donde un gen (patrón) inicia dentro de la secuencia de ADN (texto).

- **Entrada:** `texto` (String masivo), `patron` (String).
- **Restricciones:** Tiempo $O(n + m)$, Espacio $O(m)$.
- **Estrategia (KMP):**
 1. Computar arreglo LPS (π) del `patron`.
 2. Iterar sobre el `texto` usando LPS para gestionar los fallos de coincidencia sin retroceder en el texto.

Solución Problema 1: Diseño

Módulos requeridos:

1. `compute_lps(pattern)`: Retorna `list[int]`. Se encargará de comparar el patrón consigo mismo.
2. `kmp_search(text, pattern)`: Retorna `list[int]` con los índices. Manejará los punteros `i` (texto) y `j` (patrón), acumulando los resultados cuando `j == len(pattern)`.

Pseudocódigo KMP

```
Función KMP(Texto, Patrón):  
    n = longitud(Texto)  
    m = longitud(Patrón)  
    lps = ComputarLPS(Patrón)  
    j = 0, i = 0  
    resultados = []  
  
    Mientras i < n:  
        Si Patrón[j] == Texto[i]:  
            i++, j++  
        Si j == m:  
            Añadir (i - j) a resultados  
            j = lps[j - 1]  
        Sino si i < n y Patrón[j] != Texto[i]:  
            Si j > 0:  
                j = lps[j - 1]  
            Sino:  
                i++  
    Retornar resultados
```

Código Python KMP (Parte 1: Construcción)

```
def compute_lps(pattern: str) -> list[int]:
    m = len(pattern)
    lps = [0] * m
    j = 0

    for i in range(1, m):
        while j > 0 and pattern[i] != pattern[j]:
            j = lps[j - 1]

        if pattern[i] == pattern[j]:
            j += 1

        lps[i] = j

    return lps
```

Código Python KMP (Parte 2: Búsqueda)

```
def kmp_search(text: str, pattern: str) -> list[int]:
    if not pattern: return []
    lps = compute_lps(pattern)
    matches = []
    j = 0

    for i in range(len(text)):
        while j > 0 and text[i] != pattern[j]:
            j = lps[j - 1]

        if text[i] == pattern[j]:
            j += 1

        if j == len(pattern):
            matches.append(i - j + 1)
            j = lps[j - 1]

    return matches
```


Problema 2: Búsqueda Múltiple con Z-Function

Un sistema de indexación necesita **localizar las apariciones de un término** dentro de grandes documentos de texto plano. Esta vez, aplicaremos un enfoque diferente usando la *Z-Function*.

La técnica consiste en concatenar el patrón y el texto usando un carácter separador especial (por ejemplo, \$) que no exista en ninguno de los dos.

Solución Problema 2: Análisis (Concatenación)

Cadena de análisis: $S = \text{Patrón} + \$ + \text{Texto}$

Al aplicar la función Z a esta cadena combinada, estamos buscando subcadenas que coincidan con el prefijo de S . ¡Y el prefijo de S es exactamente nuestro Patrón!

Cualquier índice i donde $Z[i] == \text{len}(\text{Patrón})$ indica que, en esa posición, comienza una copia exacta del patrón original.

Solución Problema 2: Diseño

Concatenar: `string_combinado = patron + "$" + texto`.

Computar el arreglo Z del `string_combinado` manteniendo la ventana `[L, R]`.

Iterar el arreglo Z; siempre que el valor sea igual a la longitud del patrón original, calcular el índice real en el texto (restando la longitud del patrón y del separador).

Restricción: El separador `$` garantiza que el valor Z no supere la longitud del patrón.

Diagrama de Flujo Z-Function

```
graph TD
  A([Inicio Z-Function]) --> B[Z = arreglo ceros, L = 0, R = 0]
  B --> C[Para i de 1 a N-1]
  C --> D{i <= R?}
  D -- Sí --> E[Z[i] = min(R - i + 1, Z[i - L])]
  D -- No --> F[Z[i] = 0]
  E --> G[Mientras S[i + Z[i]] == S[Z[i]]]
  F --> G
  G --> H[Z[i]++]
  H --> I{i + Z[i] - 1 > R?}
  I -- Sí --> J[L = i, R = i + Z[i] - 1]
  I -- No --> K[Siguiente i]
  J --> K
  K --> C
```

Pseudocódigo Z-Function para Búsqueda

```
Función Z_Search(Texto, Patrón):  
    S = Patrón + "$" + Texto  
    N = longitud(S)  
    Z = arreglo ceros tamaño N  
    L = 0, R = 0  
    Resultados = []  
  
    Para i = 1 hasta N-1:  
        Si i <= R:  
            Z[i] = min(R - i + 1, Z[i - L])  
        Mientras i + Z[i] < N y S[Z[i]] == S[i + Z[i]]:  
            Z[i]++  
        Si i + Z[i] - 1 > R:  
            L = i  
            R = i + Z[i] - 1  
  
        Si Z[i] == longitud(Patrón):  
            Añadir (i - longitud(Patrón) - 1) a Resultados
```

Código Python Z-Function (Búsqueda)

```
def z_search(text: str, pattern: str) -> list[int]:
    if not pattern: return []

    s = pattern + "$" + text
    n = len(s)
    z = [0] * n
    l, r = 0, 0
    m = len(pattern)
    matches = []

    for i in range(1, n):
        if i <= r:
            z[i] = min(r - i + 1, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1

        if z[i] == m:
            matches.append(i - m - 1)

    return matches
```

Conclusiones Generales

- La amnesia algorítmica es el enemigo principal en el procesamiento masivo de cadenas. Omitir caracteres ya verificados destroza el rendimiento.
- El pre-procesamiento del patrón, aunque añade una pequeña sobrecarga inicial $O(m)$, transforma un problema cuadrático $O(n \times m)$ en uno estrictamente lineal $O(n + m)$.
- Tanto KMP como Z-Function aplican el principio de la programación dinámica: reutilizar estados calculados previamente.

Comparativa: KMP vs Z-Function

Característica	KMP	Z-Function
Paradigma	Prefijo = Sufijo	Prefijo = Subcadena interna
Complejidad	$O(n + m)$	$O(n + m)$
Memoria	$O(m)$	$O(n + m)$ (al concatenar)
Simplicidad	Lógica de retroceso abstracta	Intuitivo (cajas de coincidencia)
Uso Ideal	Flujos continuos (streams)	Cadenas ya cargadas en memoria

4. Tarea

Tarea: Preparación para Próxima Clase

El Problema del Palíndromo Más Largo

Diseña una solución para encontrar la subcadena palindrómica más larga dentro de un texto dado. Ejemplo: Para `babad`, la respuesta es `bab` o `aba`.

Objetivo Actual: Implementar una solución de expansión desde el centro con complejidad $O(n^2)$.

Esta solución base será nuestro punto de partida para la próxima clase.

Tips para el Ejercicio

1. Un palíndromo se lee igual de izquierda a derecha.
2. Considera que los palíndromos pueden tener longitud impar (e.g., `aba`, centro en 'b') o par (e.g., `abba`, centro entre las dos 'b').
3. Para tu solución $O(n^2)$, itera por cada carácter asumiendo que es el centro, y usa dos punteros (izquierdo y derecho) expandiéndote hacia afuera mientras los caracteres coincidan.

Lecturas para la Próxima Clase

En la próxima sesión superaremos el límite de $O(n^2)$ usando una lógica de simetría similar a la de hoy. Estudiaremos el **Algoritmo de Manacher**, que logra encontrar el palíndromo más largo en tiempo lineal estricto $O(n)$.

Asignaciones:

- Investigar qué son los *falsos centros* en la búsqueda de palíndromos.
- Leer sobre la inyección de caracteres de separación (ej. `#a#b#a#`) para estandarizar palíndromos pares e impares.